Pokhara University
Faculty of Science and Technology

Course No.: CMP 515 (3 Credits)	Full marks: 100
Course title: Operating System Design and Implementation	Pass marks: 60
Nature of the course: Theory	Total Lectures: 45 hrs
Level: Master	Program: ME/MSc (Computer)

1. Course Description

This course explores advanced concepts and techniques in operating system design and implementation, including process management, memory organization, file systems, virtualization, and security. Emphasizing both theory and practical application, students will design OS modules, analyze performance, and explore emerging trends like cloud OS, real-time systems, and unikernels.

2. General Objectives

After successful completion of this course, students should be able to:

- Gain in-depth knowledge of operating system design principles and implementation techniques.
- Analyze and optimize advanced concepts such as concurrency, resource management, and security.
- Develop critical thinking for solving real-world OS-related problems.
- Research emerging trends and advancements in operating systems.

3. Methods of Instruction

Lecture, Discussion, Readings, Case Studies, Project and Research Works.

4. Contents in Detail

Specific Objectives	Contents

• Explain the concepts of OS, Kernel, its design and implementation.	**1. Operating System Architecture (4 hrs)** 1.1. Review on operating system: OS concepts, System Calls, Process, Threads, CPU Scheduling, Memory, Storage, I/O Management. 1.2. Operating System Design Goals and Principles 1.3. Operating System Structures and Kernel Types: Layered, Monolithic, Microkernel, Hybrid Kernel, Exokernel, Nanokernel, and Modular Kernel 1.4. Kernel Design and Implementation Strategies 1.5. Case Studies : Linux Kernel Architecture, Windows NT Architecture
• Discuss the advanced process scheduling algorithms in multicore and NUMA systems. • Explain the advanced process and thread synchronization techniques.	**2. Advanced Process and Thread Management (10 hrs)** 2.1. Advanced Process Scheduling Techniques: Multi-core (Affinity based, Load Balancing, Cache and memory Optimization) and NUMA-aware scheduling 2.2. Thread Libraries and Synchronization: Real-time (Semaphore-based, conditional variable, synchronization protocols) and Adaptive synchronization techniques (Adaptive locks, lock free data structure, synchronization protocol) 2.3. Deadlock Detection Algorithms (Centralized, Distributed and Hierarchical) 2.4. Distributed Mutual Exclusion Algorithms (Lamports, Token and Non Token Based Ricart Agarwala, Epidemic Algorithms, Maekawa's) 2.5. Case Studies: Process Management in Linux and Windows.
• Explain the advanced virtual memory techniques and advanced page replacement algorithms • Describe the core principles behind memory virtualization and its role in modern OSes.	**3. Memory Management and Virtualization (8 hrs)** 3.1. Advanced Virtual Memory: Huge Pages, Transparent Huge Pages 3.2. Advanced Page Replacement Algorithms: Clock-pro and WSClock. 3.3. Cache-aware Memory Management 3.4. Principles for Virtualization 3.5. Hypervisor Architectures 3.6. Virtualization of Memory and I/O in Multi-Core Systems 3.7. Memory Isolation and Protection in Virtualized Environments.

	3.8. Case studies: Memory Management in Linux, Virtualization Technologies (ZEN, VMware and AMD-V)
• Describe the distributed file systems, storage technology and modern file systems.	**4. File System and Storage Management (6 hrs)** 4.1. Distributed File Systems: HDFS, Ceph, Google File System 4.2. Storage Technologies: SSDs, NVM, Persistent Memory. 4.3. Modern File Systems: ZFS, Btrfs, EXT4, APFS. 4.4. Scalability and Consistency Challenges in file and storage.. 4.5. Case Studies: DNA Storage, Holographic Storage, Quantum Storage
• Learn and apply how modern OSes handle and mitigate side-channel attacks for improved security.	**5. Security and Isolation in Operating Systems (6 hrs)** 5.1. Kernel Security Mechanisms: SELinux, AppArmor. 5.2. Isolation Techniques: Namespaces, Cgroups, and Sandboxing. 5.3. Address Space Layout Randomization (ASLR) and Stack Protection. 5.4. Mitigation Techniques for Side-channel Attacks. 5.5. Case Studies: Linux Security Module, Android Security Architecture, macOS Security Architecture)
• Describe how embedded and real-time OSes optimize energy consumption for battery-powered devices.	**6. Real-Time and Embedded Operating Systems (6 hrs)** 6.1. Real-Time Scheduling Algorithms (EDF, RMS, RR,LLF) 6.2. Embedded OS Features: TinyOS, FreeRTOS 6.3. Energy-efficient OS Design 6.4. Case Study: Android and Embedded Linux
• Describe the concepts of distributed OS and cloud OS along with its platforms.	**7. Distributed and Cloud Operating Systems (5 hrs)** 7.1. Key concepts in Distributed Operating System Architecture (Transparency, Communication, Synchronization). 7.2. Cloud OS: Kubernetes, OpenStack. 7.3. Fault Tolerance and Load balancing in distributed systems: Concepts, types, techniques and algorithms. 7.4. Resource Management in Cloud Environments.

	7.5. Case studies : Amazon DynamoDB (NoSQL), Microsoft Azure and Fault Tolerant DS.

5. Evaluation System and Students' Responsibilities

Evaluation System

The internal evaluation of a student may consist of assignments, attendance, internal assessment, project and research works etc. The internal evaluation scheme for this course is as follows (but not limited to):

Internal Evaluation	**Weight**	**Marks**	**External Evaluation**	**Marks**
Attendance & Class Participation	10%	60	Semester-End examination	40
Assignments	20%			
Internal Assessment	40%			
Research/Project/Case Study Work	30%			
Total Internal		60	**Total External**	40
Full Marks: 60 + 40 = 100				

Student Responsibilities

Each student must secure at least 60% marks separately in internal evaluation with 80% attendance in the class in order to appear in the Semester End Examination. Failing to get such a score will be given NOT QUALIFIED (NQ) to appear for the Semester-End Examinations. Students are advised to attend all the classes, formal exam, test, etc. and complete all the assignments within the specified time period. Students are required to complete all the requirements defined for the completion of the course.

6. Reference Books

1. Operating Systems: Three Easy Pieces by Remzi H. Arpaci-Dusseau.
2. Modern Operating Systems by Andrew S. Tanenbaum.
3. Understanding the Linux Kernel by Daniel P. Bovet and Marco Cesati.
4. Abraham Silberschatz, Peter Baer Galvin and Greg Gagne, Operating System Concepts, 10th Edition, Wiley, 2018
5. Linux Kernel Development by Robert Love (3rd or 4th Edition)
6. Real-Time Systems by Jane W. S. Liu
7. Distributed Systems: Concepts and Design by George Coulouris, Jean Dollimore, Tim Kindberg, and Gordon Blair